

Analizador sintáctico

Gramática del lenguaje de pseudocódigo

<Programa> -> inicio-programa <Enunciados> fin-programa

<Enunciados> -> <Enunciado> <Enunciados> | <Enunciado>

<Enunciado> -> <Asignacion> | <Leer> | <Escribir> | <Si> | <Mientras>

<Asignacion> -> VARIABLE = <Expresion>

<Expresion> -> <Valor> <Operador aritmetico> <Valor> | <Valor>

<Leer> -> leer VARIABLE

<Escribir> -> escribir CADENA | escribir VARIABLE | escribir CADENA , VARIABLE

<Si> -> si <Comparacion> entonces <Enunciados> fin-si

<Mientras> -> mientras <Comparacion> <Enunciados> fin-mientras

<Comparacion> -> (<Valor> <Operador relacional> <Valor>)

<Valor> -> VARIABLE | NUMERO

<Operador aritmetico> -> + | - | * | /

<Operador relacional> -> , | > | <= | >= | == | !=

```

public class PseudoParser {
    private ArrayList<Token> tokens;
    private int indiceToken = 0;
    private SyntaxException ex;

    public void analizar(PseudoLexer lexer) throws SyntaxException {
        tokens = lexer.getTokens();

        if (Programa()) {
            if (indiceToken == tokens.size()) {
                System.out.println("\nLa sintaxis del programa es correcta");
                return;
            }
        }

        throw ex;
    }

    // <Programa> -> inicio-programa <Enunciados> fin-programa
    private boolean Programa() {
        if (match("INICIOPROGRAMA"))
            if (Enunciados())
                if (match("FINPROGRAMA"))
                    return true;

        return false;
    }

    // <Enunciados> -> <Enunciado> <Enunciados>
    private boolean Enunciados() {
        int indiceAux = indiceToken;

        if (Enunciado()) {
            while (Enunciado());
            return true;
        }

        indiceToken = indiceAux;
        return false;
    }
}

```

```
// <Enunciado> -> <Asignacion> | <Leer> | <Escribir> | <Si> | <Mientras>
private boolean Enunciado() {
    int indiceAux = indiceToken;

    if (tokens.get(indiceToken).getTipo().getNombre().equals("VARIABLE"))
    |   if (Asignacion())
    |       return true;

    indiceToken = indiceAux;

    if (tokens.get(indiceToken).getTipo().getNombre().equals("LEER"))
    |   if (Leer())
    |       return true;

    indiceToken = indiceAux;

    if (tokens.get(indiceToken).getTipo().getNombre().equals("ESCRIBIR"))
    |   if (Escribir())
    |       return true;

    indiceToken = indiceAux;

    if (tokens.get(indiceToken).getTipo().getNombre().equals("SI"))
    |   if (Si())
    |       return true;

    indiceToken = indiceAux;

    if (tokens.get(indiceToken).getTipo().getNombre().equals("MIENTRAS"))
    |   if (Mientras())
    |       return true;

    indiceToken = indiceAux;
    return false;
}
```

```
// <Asignacion> -> VARIABLE = <Expresion>
```

```
private boolean Asignacion() {  
    int indiceAux = indiceToken;  
  
    if (match("VARIABLE"))  
        if (match("IGUAL"))  
            if (Expresion())  
                return true;  
  
    indiceToken = indiceAux;  
    return false;  
}
```

```
// <Expresion> -> <Valor> <Operador aritmetico> <Valor> | <Valor>
```

```
private boolean Expresion() {  
    int indiceAux = indiceToken;  
  
    if (Valor())  
        if (match("OPARITMETICO"))  
            if (Valor())  
                return true;  
  
    indiceToken = indiceAux;  
  
    if (Valor())  
        return true;  
  
    indiceToken = indiceAux;  
    return false;  
}
```

```
// <Valor> -> VARIABLE | NUMERO
```

```
private boolean Valor() {  
    if (match("VARIABLE") || match("NUMERO"))  
        return true;  
  
    return false;  
}
```

```
// <Leer> -> leer VARIABLE
private boolean Leer() {
    int indiceAux = indiceToken;

    if (match("LEER"))
        if (match("VARIABLE"))
            return true;

    indiceToken = indiceAux;
    return false;
}
```

```
// <Escribir> -> escribir CADENA , VARIABLE | escribir CADENA | escribir VARIABLE
private boolean Escribir() {
    int indiceAux = indiceToken;

    if (match("ESCRIBIR"))
        if (match("CADENA"))
            if (match("COMA"))
                if (match("VARIABLE"))
                    return true;

    indiceToken = indiceAux;

    if (match("ESCRIBIR"))
        if (match("CADENA"))
            return true;

    indiceToken = indiceAux;

    if (match("ESCRIBIR"))
        if (match("VARIABLE"))
            return true;

    indiceToken = indiceAux;
    return false;
}
```

```

// <Si> -> si <Comparacion> entonces <Enunciados> fin-si
private boolean Si() {
    int indiceAux = indiceToken;

    if (match("SI"))
        if (Comparacion())
            if (match("ENTONCES"))
                if (Enunciados())
                    if (match("FINSI"))
                        return true;

    indiceToken = indiceAux;
    return false;
}

// <Mientras> -> mientras <Comparacion> <Enunciados> fin-mientras
private boolean Mientras() {
    int indiceAux = indiceToken;

    if (match("MIENTRAS"))
        if (Comparacion())
            if (Enunciados())
                if (match("FINMIENTRAS"))
                    return true;

    indiceToken = indiceAux;
    return false;
}

// <Comparacion> -> ( <Valor> <Operador relacional> <Valor> )
private boolean Comparacion() {
    int indiceAux = indiceToken;

    if (match("PARENTESISIZQ"))
        if (Valor())
            if (match("OPRELACIONAL"))
                if (Valor())
                    if (match("PARENTESISDER"))
                        return true;

    indiceToken = indiceAux;
    return false;
}

```

```

private boolean match(String nombre) {
    if (tokens.get(indiceToken).getTipo().getNombre().equals(nombre)) {
        System.out.println(nombre + ": " +
                            tokens.get(indiceToken).getNombre());

        indiceToken++;
        return true;
    }

    ex = new SyntaxException("Se esperaba: '" + nombre +
                            "' y se encontró: '" +
                            tokens.get(indiceToken).getTipo().getNombre() +
                            "'");
    return false;
}
}

```

```

public class SyntaxException extends Exception {
    public SyntaxException(String message) {
        super(message);
    }

    public SyntaxException(String message1, String message2) {
        super("Se esperaba un token '" + message1 +
            "' y se encontró '" + message2 + "'");
    }
}

```



```

public class PruebaParser {
    public static void main(String[] arg) throws LexicalException, SyntaxException {
        String entrada = leerPrograma("/.../ejemplo.alg");
        PseudoLexer lexer = new PseudoLexer();
        lexer.analizar(entrada);

        System.out.println("*** Análisis léxico ***\n");

        for (Token t: lexer.getTokens()) {
            System.out.println(t);
        }

        System.out.println("\n*** Análisis sintáctico ***\n");

        PseudoParser parser = new PseudoParser();
        parser.analizar(lexer);
    }

    private static String leerPrograma(String nombre) {
        String entrada = "";

        try {
            FileReader reader = new FileReader(nombre);
            int character;

            while ((character = reader.read()) != -1)
                entrada += (char) character;

            reader.close();
            return entrada;
        } catch (IOException e) {
            return "";
        }
    }
}

```

```

inicio-programa
    leer numeroDeElementos
    promedio = 0
    i = 0

    mientras (i < numeroDeElementos)
        leer elemento
        promedio = promedio + elemento
        i = i + 1
    fin-mientras

    promedio = promedio / numeroDeElementos
    escribir "El promedio es ", promedio
fin-programa

```

```

*** Análisis sintáctico ***

INICIOPROGRAMA: inicio-programa
LEER: leer
VARIABLE: numeroDeElementos
VARIABLE: promedio
IGUAL: =
NUMERO: 0
NUMERO: 0
VARIABLE: i
IGUAL: =
NUMERO: 0
NUMERO: 0
MIENTRAS: mientras
PARENTESISIZQ: (
VARIABLE: i
OPRELACIONAL: <
VARIABLE: numeroDeElementos
PARENTESISDER: )
LEER: leer
VARIABLE: elemento
VARIABLE: promedio
IGUAL: =
VARIABLE: promedio
OPARITMETICO: +
VARIABLE: elemento
VARIABLE: i
IGUAL: =
VARIABLE: i
OPARITMETICO: +
NUMERO: 1
FINMIENTRAS: fin-mientras
VARIABLE: promedio
IGUAL: =
VARIABLE: promedio
OPARITMETICO: /
VARIABLE: numeroDeElementos
ESCRIBIR: escribir
CADENA: "El promedio es "
COMA: ,
VARIABLE: promedio
FINPROGRAMA: fin-programa

La sintaxis del programa es correcta

```

Ejercicio

- En el primer ejercicio del analizador léxico se extendió el lenguaje de pseudocódigo para permitir la declaración de las variables utilizadas en un programa y se modificó la clase PseudoLexer para detectar las nuevas unidades léxicas en la declaración. Un enunciado de declaración de variables sería como el que se muestra:
 - **variables: numeroDeElementos, promedio, i**
- Las primeras producciones (reglas de sintaxis) de la gramática actual del lenguaje de pseudocódigo son las siguiente:
 - <Programa> -> inicio-programa <Enunciados> fin-programa
 - <Enunciados> -> <Enunciado> <Enunciados> | <Enunciado>
 - <Enunciado> -> <Asignacion>|<Leer>|<Escribir>|<Si>|<Mientras>
 - ...
- ¿Cómo se debe modificar la gramática para incluir en el lenguaje de pseudocódigo la declaración de variables?
- Modifica la clase **PseudoParser** y las clases relacionadas que se requieran para permitir la declaración de variables en el lenguaje de pseudocódigo.

Ejercicio

- En el segundo ejercicio del analizador léxico se extendió el lenguaje de pseudocódigo para permitir la utilización del enunciado **repite** como estructura de iteración y se modificó la clase **PseudoLexer** para detectar las nuevas unidades léxicas en el nuevo enunciado. Un ejemplo del uso del enunciado repite puede ser:

repite (i, 0, numeroDeElementos)

leer elemento

promedio = promedio + elemento

fin-repite

- El enunciado repite requiere tres elementos: una variable que se utiliza como índice, el valor inicial y el valor final para la variable índice.
- ¿Cómo se debe modificar la gramática para extender el lenguaje de pseudocódigo con el enunciado **repite**?
- Modifica la clase **PseudoParser** y las clases relacionadas que se requieran para permitir el uso del enunciado **repite**.